Propshaft

Bundling the JavaScript and CSS files together is only half of the process of serving assets, sometimes called the *asset pipeline*. You also have to send the asset files down to the server. This would seem to be straightforward—just put the files in the public directory—but there are always ways to make things a little more interesting in the name of performance or ease of use.

Pre-Rails 7, the default asset pipeline was the Sprockets gem. Sprockets provided both the bundling and asset delivery services, allowing you to create a manifest of the JavaScript and CSS files in your bundle, bundling them, and then providing them to the browser with a few extra features to manage caching and whatnot. Alternately, Webpacker provided the same features using webpack as the build tool.

It was a useful tool, but the Rails team struggled to keep Sprockets up to date with the larger front-end ecosystem. With the advent of the jsbundling and cssbundling gems, there's no need to keep up, the integration point has moved, and the third-party tool is now responsible for everything needed to put the bundle into the `assets/builds` directory. This allows for an asset pipeline tool that has much less responsibility and is significantly simpler than Sprockets, which is where Propshaft comes in.

Propshaft does basically four things, and it does them more or less transparently once installed:

- In development, Propshaft sets itself up as a server for assets so that the assets do not need to be explicitly managed. In production, Propshaft uses the same `asset:precompile` Rake task that Sprockets used.

- Propshaft adds a digest hash to each file name so that files are cached by your server until changed. Your code can still refer to the file by its regular name without the digest. In development, the development server manages this. In production, the precompile step provides a manifest file that the